

Notes d'analyse : Projet JGrapa

Table des matières

1	Compréhension du projet	2
1.1	Objectif principal	2
1.2	Caractéristiques fondamentales	2
2	Dossier Assessment : Structure des évaluations	2
2.1	Analyse des fichiers et des classes	2
2.2	Liens et architecture	3
3	Dossier Aggregator : Stratégies de calcul	3
3.1	Analyse des fichiers et des classes	3
3.2	Liens et architecture interne du module	4
4	Interactions globales : Synthèse des liaisons	4
4.1	Le point de convergence : <code>Exam.java</code>	4
4.2	Séquence de liaison et de calcul	4

1 Compréhension du projet

1.1 Objectif principal

Le projet JGrapa (Java Grade Parser) est un moteur de modélisation et de calcul de notes. Son objectif fondamental est de séparer strictement le stockage des données d'évaluation (les notes brutes et les commentaires) de la logique de calcul (le barème et les règles mathématiques). Cette séparation permet d'appliquer différentes stratégies d'évaluation sur une même copie sans jamais altérer la structure des notes d'origine.

1.2 Caractéristiques fondamentales

Le fonctionnement de JGrapa repose sur les principes et contraintes suivants :

- **Découplage** : La donnée de l'étudiant (*Assessment*) et la méthode de calcul du professeur (*Aggregator*) sont deux entités indépendantes. Elles ne sont réunies qu'au niveau final (*Exam*) pour produire un résultat.
- **Structure en arborescence** : Les évaluations sont organisées sous forme d'arbre. Cette disposition (dossiers, sous-dossiers, matières) sert à regrouper les notes de manière logique, avant même de définir comment elles seront calculées.
- **Bornes mathématiques strictes** : Une note valide est toujours un nombre décimal compris entre -1.0 et 1.0. Les poids (coefficients) doivent être positifs et leur somme locale ne doit jamais dépasser 1.0 (à une marge de tolérance informatique de 10^{-9} près).
- **Multiplicité des algorithmes de calcul** : Le moteur ne se limite pas à la moyenne pondérée classique. Il gère :
 - Des pondérations statiques (coefficients fixes par matière).
 - Des pondérations basées sur le rang/tri de l'étudiant (algorithme OWA - par exemple, donner plus de poids à la meilleure note).
 - Des pondérations conditionnelles (algorithme paramétrique - où la note d'un critère influence le coefficient des autres).
- **Persistance par schémas** : L'importation et l'exportation des barèmes et des copies s'effectuent via des fichiers JSON. La validité de ces fichiers est garantie par des JSON Schemas stricts qui vérifient la conformité des données avant leur traitement en mémoire.
- **Immutabilité** : Les structures de données créées (arbres de notes, dictionnaires de coefficients) sont scellées et non modifiables une fois instanciées.

2 Dossier Assessment : Structure des évaluations

Ce module gère exclusivement le stockage et l'organisation en mémoire des notes brutes et des retours textuels, sans intégrer aucune logique de calcul.

2.1 Analyse des fichiers et des classes

- **Mark.java** : C'est l'unité de notation. Il s'agit d'un *record* (une structure de données purement porteuse de variables) qui encapsule une valeur de type **double**. Sa règle de construction est stricte : la note doit obligatoirement se situer dans l'intervalle $[-1.0, 1.0]$. La classe inclut des méthodes pour instancier rapidement la note maximale (**max()**) ou minimale (**min()**).
- **AssessmentTree.java** : C'est le contrat de base de l'arborescence. Il s'agit d'une interface scellée (*sealed interface*). Elle verrouille la structure de l'arbre en n'autorisant que deux implémentations possibles et définitives : **Assessment** (la feuille) et **CompositeAssessmentTree**

(le nœud).

- **Assessment.java** : Il s'agit d'une implémentation de **AssessmentTree**. Elle représente la fin d'une branche de l'arbre (la copie corrigée). Cette classe stocke exactement deux informations indissociables : un objet **Mark** (la note numérique) et un **String** (le **feedback** ou commentaire de correction). Si le correcteur ne fournit pas de texte, le système force une chaîne de caractères vide par sécurité.
- **CompositeAssessmentTree.java** : Il s'agit de la seconde implémentation de **AssessmentTree**. Elle représente un regroupement ou un dossier (par exemple, "Semestre 1" ou "Pôle Scientifique"). Contrairement à **Assessment**, elle ne contient ni note ni commentaire. Elle possède un unique attribut : un dictionnaire immuable (**ImmutableMap**) qui associe des clés de type **Criterion** (le nom de la matière) à des valeurs de type **AssessmentTree** (les enfants du dossier).
- **AssessmentTreeJsonConverter.java** : Elle contient les instructions pour transformer les objets Java de ce dossier en une chaîne de texte JSON (sérialisation), et l'inverse (désérialisation). Elle s'appuie sur le schéma **assessment-tree.schema.json** pour valider le format des données entrantes avant d'autoriser la création des objets en mémoire.

2.2 Liens et architecture

La mécanique interne fonctionne par récursivité, articulée autour de l'interface parent **AssessmentTree**. Un objet **CompositeAssessmentTree** ignore la nature exacte de ses enfants. Son dictionnaire pointe vers des **AssessmentTree** génériques.

Concrètement, l'enfant associé à un critère dans le dictionnaire peut être soit une évaluation finale (**Assessment**), soit un autre sous-dossier (**CompositeAssessmentTree**). Ce système permet de construire des arbres de n'importe quelle profondeur et asymétrique.

La chaîne de dépendance est descendante : la validité d'un arbre **CompositeAssessmentTree** dépend de la validité de ses branches, qui doivent toutes inévitablement se terminer par des feuilles de type **Assessment**. Celles-ci dépendent à leur tour de la conformité de l'objet **Mark** qu'elles contiennent.

3 Dossier Aggregator : Stratégies de calcul

Ce module est exclusivement dédié à la logique mathématique. Il définit les règles de pondération appliquées aux notes. Il ne stocke aucune donnée d'évaluation.

3.1 Analyse des fichiers et des classes

- **Aggregator.java** : Classe abstraite scellée servant de contrat de base pour toutes les méthodes de calcul. Elle autorise uniquement trois implémentations : **Weighter**, **Owa** et **Parametric**. Elle gère la structure de l'arbre de calcul via un dictionnaire de sous-agrégateurs (**subs**) et un agrégateur par défaut (**defaultSub**).
- **Aggregator.WeightedMarks** : Classe interne statique qui associe une copie aplatie (**OneLevelMarksTree**) à un dictionnaire de poids dynamiques. Son constructeur valide trois conditions strictes : poids finis, poids positifs, et somme des poids n'excédant pas 1.0 (à la tolérance 10^{-9} près). Elle expose la méthode **weightedSum()** qui exécute l'opération mathématique.
- **Weighter.java** : Implémentation de la moyenne pondérée statique. Les poids sont fixés à l'avance pour chaque critère. Si une copie d'étudiant ne contient pas tous les critères prévus par le barème, la classe recalcule les poids restants pour répartir la proportion manquante sur les notes présentes, maintenant ainsi un total de 1.0.

- **Owa.java** : Implémentation de la méthode *Ordered Weighted Averaging*. Les poids ne sont pas associés à des critères, mais à un rang de classement. La classe trie les notes de l'étudiant de manière décroissante avant d'appliquer les coefficients. Elle inclut une logique d'ajustement (duplication ou filtrage) si le nombre de notes diffère du nombre de poids fournis.
- **Parametric.java** : Implémentation d'un barème conditionnel liant deux critères spécifiques : un critère à multiplier (**multiplied**) et un critère de pondération (**weighting**). La note obtenue au critère de pondération écrase et détermine la valeur du coefficient appliqué au critère multiplié et répartit le reste du poids sur les matières annexes.
- **AggregatorJsonConverter.java** : Utilitaire de sérialisation et de désérialisation. Il instancie les objets **Aggregator** à partir de fichiers JSON et vérifie leur intégrité syntaxique à l'aide du schéma **aggregator.schema.json**.

3.2 Liens et architecture interne du module

L'architecture du module repose sur le patron de conception **Stratégie**. La classe mère **Aggregator** délègue la création des coefficients à ses classes filles.

Le comportement interne est uniformisé : chaque classe (**Weighter**, **Owa**, **Parametric**) implémente la méthode abstraite **aggregate(OneLevelMarksTree)**. La tâche de cette méthode n'est pas de calculer la moyenne finale, mais de générer un dictionnaire de coefficients (statiques ou recalculés dynamiquement) correspondant aux notes de l'étudiant.

Une fois le dictionnaire généré, il est transmis à la classe **WeightedMarks**. L'agrégateur prépare les variables de l'équation, mais ne fait jamais l'addition lui-même.

4 Interactions globales : Synthèse des liaisons

Le système est conçu pour que les objets d'évaluation (**Assessment**) et les comportements mathématiques (**Aggregator**) ne se rencontrent que dans le module **Exam**.

4.1 Le point de convergence : Exam.java

La classe **Exam.java** est le conteneur global. Elle lie un identifiant d'étudiant (**StudentId**) à son arbre de notes (**AssessmentTree**), et associe l'ensemble des copies à une unique racine d'agrégation (**Aggregator**).

4.2 Séquence de liaison et de calcul

Le calcul d'une note finale suit une chaîne de dépendance stricte entre les deux dossiers :

1. **Extraction (Assessment → Aggregator)** : Les objets **Assessment** contenant les instances de **Mark** sont extraits des dossiers **CompositeAssessmentTree**. Ils sont convertis en une structure plate nommée **OneLevelMarksTree** pour annuler la profondeur de l'arbre lors du calcul d'un niveau spécifique.
2. **Distribution des rôles** : Le dictionnaire **subs** de la classe **Aggregator** attribue le moteur de calcul approprié à chaque sous-dossier de la copie.
3. **Génération de l'équation** : L'agrégateur sélectionné reçoit le **OneLevelMarksTree**. Il lit les objets **Criterion** et leurs valeurs **Mark**, applique sa logique interne (tri, statique ou conditionnelle) et génère un dictionnaire **ImmutableMap<Criterion, Double>**.

4. **Validation croisée** : L'objet `OneLevelMarksTree` (les données) et le dictionnaire de poids (les règles) sont injectés dans `WeightedMarks`. Ce dernier vérifie formellement que l'ensemble des clés `Criterion` est identique dans les deux structures.
5. **Résolution** : La méthode `weightedSum()` est invoquée. Elle extrait la valeur (`double`) contenue dans `Mark`, la multiplie par le `Double` du dictionnaire de poids, incrémente le résultat et retourne la note calculée.

Les classes du dossier `assessment` ignorent l'existence des algorithmes, et les classes du dossier `aggregator` manipulent l'évaluation de l'étudiant via des interfaces génériques sans en modifier les données internes.